Response to reviewers

Manuscript: GENOME/2026/282393 **Original title:** *Evaluating open LLMs for agentic analysis orchestration in a typical biomedical lab* **Revised title:** *In a single pass, most local 4-bit executors tested transcribe a detailed plan; two of thirteen bind it* **Decision:** Return for Revision (22 July 2026)

Revised files: `writeup_R2.md` (main text) and `supplement_R2.md` (Supplementary Methods, Results, Discussion, Tables S1–S20, Figures S1–S4). Every passage cited below is present in those files. The source files map each passage to a review point with " comments. Any claim in this letter can therefore be checked against the text it refers to.

---

1. Covering statement

All three reviewers converged on two objections. We accept both and did not argue with either.

The system was not agentic, and the word is gone from the title. Benchmark 1 is one prompt in, one script out, executed externally, with no observation and no repair. That is plan-conditioned code generation, not orchestration. The title now says what was measured. The Introduction states it in the paper's own voice: "Benchmark 1's harness is not an agent harness at all — one prompt in, one script out, executed externally, with no loop, no observation and no repair — which isolates plan detail as the only variable but means the system should not be called agentic".

One workflow was not enough, so a second workflow class was run, on a shared public production server, in a real tool-using loop with observation and durable state: per-sample RNA-seq quantification of *Candidozyma auris* (PRJNA904261) through the 30-step IWC `rnaseq-pe` workflow on usegalaxy.org. It is reported as a case study, not an experiment, because it is one invocation per arm with no replicates and no frontier control.

Four results are new. We ask the reviewers to weigh these four first.

1. **The transcription-versus-binding question was answered by experiment. For most models the answer confirms Reviewer 2's objection.** After the second review cycle we ran the perturbed-plan control. This letter previously listed that control as the paper's single most valuable missing experiment. The setup: plan v2 verbatim; the reference genome moved to `data/ref/GRCh38_chrM/rCRS.fa`; the real path stated in the prompt's DATASET section. Before any model ran, the sandbox was validated with a hand-written copier (fails) and binder (succeeds). Thirteen local models × 3 seeds gave **6/39 perfect against a 34/36 control**. The result is bimodal by model: **every model is 3/3 or 0/3**. Eleven of thirteen models copied the plan's stale path verbatim and died at the first step. The original headline model `qwen3.6:27b` is among the eleven. Two models (`gemma4:31b`, `gpt-oss:20b`) rebound every path and stayed perfect. For eleven of thirteen local models, v2 success is transcription. Binding exists in this class but is rare. The title, abstract, Results and Discussion now carry this as the central result.

2. **Executable syntax, rather than plan length, accounts for most of the observed plan effect.** Stripping every word of explanation from the detailed plan (v1.5) reduced perfect runs from 34/36 to 30/36; paired by model, 3 improved, 1 worsened and 8 were unchanged (two-sided sign-test $p = 0.63$). Removing its command lines reduced perfect runs to 9/36. At up to ten seeds per model, a plan stating `lofreq call-parallel` as one runnable line scored 63/127, whereas a registry-derived plan expressing the invocation in a non-runnable six-line form scored 31/126. Because repeated seeds are clustered within models, we replaced the pooled Fisher test with an exact sign-flip test on the 13 paired model-level proportions: mean within-model difference 0.253, two-sided $p = 0.027$. The all-code plan scored 109/127; its mean within-model advantage over the one-line plan was 0.354 ($p = 0.0039$). The moved-path experiment then showed literal transcription directly in eleven of thirteen models. The manuscript now distinguishes that evidence from the unresolved size of any prose effect.

3. **We measured how much of the result is copying rather than asserting it is not.** The

transcription index is token-level recall of plan v2's 90 literal command tokens, computed over 514 archived scripts. Correct pipelines built with *no plan* score 0.505. Plan-conditioned correct scripts at v2 score 0.749. So about half of the detailed plan's command tokens are what any correct pipeline contains regardless of plan. Roughly a further quarter is transcription. The perturbed-plan experiment (item 1) then separated transcription from binding directly.

4. **The paper's original economic motivation fails on the paper's own numbers.** Measured API cost is \$0.0662 per run. Measured local wall time is 86.7 s per run, corresponding to \$0.00016 of marginal electricity. The annualised crossover is 25,475 runs/year with no API-side labour, 17,900 with 4 h/year and 10,325 with 8 h/year. At 16 h/year, the assumed API-side fixed labour cost already exceeds the annualised local fixed cost, so no positive crossover exists. The single-pass task measured here therefore does not justify local hardware on cost alone.

Four further changes are structural. The n = 3 headline did not survive repeated sampling at n = 10, and the headline is now n = 10. Every model that moved, moved down. The three discriminating conditions v1g, v1.25 and v1.5 were subsequently also raised to ten seeds per model (273 new cells). That extension turned the copy-pasteability contrast from a hedge into a measurement. Conventional variant-calling metrics were added, and they are informative for a reason that hurts. Precision is 1.000 *by construction* because the truth set is the same caller's output, so the metrics measure plumbing, not calling accuracy. The single-pass recovery rate under injected failure is 12.5%, not the 41.7% a pooled figure would give, because two of seven injected "errors" inject no error. And neither Galaxy executor completed the reporting step: a human read the per-sample counts out of the server.

We have also removed material rather than only adding it. The model catalogue, the agent-harness capability table and the six-class workflow table left the main text. In each case the rows were either uncheckable or author opinion.

---

2. Summary of dispositions

Consensus items

ID	Issue	Disposition
C1	Not an agentic loop	**Accepted.** "Agentic" removed from title; benchmark 1 explicitly described as non-agentic; benchmark 2 added to supply the loop. Bounded repair arm **run** at the perturbed condition: three of eleven transcribers repair when shown the error, eight do not (§5.1).
C2	One small workflow; conclusions broader than evidence	**Accepted.** Second workflow class run as a case study; claims narrowed to tasks with no data-dependent parameter choices; six-class scope table moved to Supplement with qualitative rows marked.
C3	Jaccard alone is the wrong metric	**Accepted.** Precision/recall/F1 and TP/FP/FN added (Table S12); pass/fail ladder M1/M2/M3 defined in one place; degeneracy of the local M3 stated outright.
C4	"Seed" does not control determinism	**Accepted.** All determinism language rewritten; API integer identified as a run identifier; a new `top_k`/`top_p` confound disclosed.
C5	Missing related work and baseline	**Accepted in part.** Prior art cited and originality claim dropped; Biomni, BixBench, BioMaster, BioAgents positioned. Head-to-head **not run**; argued instead.

Reviewer 1

ID	Issue	Disposition
R1.1	Security guardrails absent	**Accepted.** Guardrails documented; unsandboxed status stated; lexical audit of 766 scripts replaces the "nothing observed" claim; one genuinely out-of-sandbox action reported.
R1.2	Prior art for planner–executor	**Accepted.** Six primary citations added; architectural originality claim removed.
R1.3	Generalization and production	**Accepted.** Production concerns section added (Table S20); workflow-class taxonomy added (Table S19); no row claimed as tested.
R1.4	Data governance; planner prompt detail	**Accepted, with a finding against us.** Payload audit run; the boundary held for the planner only — the frontier *executor* arms carried subject labels across it. All prompts and two plan files reproduced verbatim.

Reviewer 2

ID	Issue	Disposition
R2.1	Reads as an implementation guide, not a paper	**Accepted, and the objection was then confirmed by experiment.** Introduction reframed around the laboratory problem; the anti-copy control R2's objection implies **was run** after the second cycle and confirmed transcription for 11 of 13 local models, with binding in 2.
R2.2	Move Table 1 to the supplement; drop reasoning/coding columns	**Accepted.** Main Table 1 is now a four-model selection; full 14-model catalogue is Table S8; both columns dropped.
R2.3	Add MoE, KV-cache, agent-loop background to main text	**Accepted.** New Introduction paragraph; agent-harness anatomy in Introduction and Supplement.
R2.4	Task taxonomy	**Accepted.** Six-class taxonomy (Table S19), with the two classes run identified as the two in which a plan can state the answer.
R2.5	Survey of open-source harnesses	**Accepted in altered form.** Narrative survey retained; the capability table was **deleted** because every cell answered yes and none was checkable.
R2.6	Sampling parameters, not seeds	**Accepted, and it got worse.** A previously undisclosed `top_k`/`top_p` confound is now reported as a confound; the pinned re-run was **not done**.
R2.7	Promote error handling to Results	**Accepted.** Error injection is now a Results section with the mechanism described and recovery rescored by class.
R2.8	Align scoring with published practice	**Accepted in part; premise contested.** Table S5 maps each choice to the nearest published convention. We do not accept that a standard rubric exists for this task shape.

ID	Issue	Disposition
R2.9	Related work; Biomni	**Accepted; head-to-head declined.** Positioned in Supplement; **no external anchor was obtained**, stated as the sharpest positioning gap.
R2.10	Single-pass generation is not agentic	**Accepted.** Same disposition as C1.
R2.11	Jaccard degenerate; state pass/fail	**Accepted.** Local M3 shown to be binary and reported as counts; pass/fail ladder defined once in Methods.
R2.12–R2.22	Detail items carried in the source markup	See §4.5.

Reviewer 3

ID	Issue	Disposition
R3.preamble	Success at the detailed end may be transcription	**Conceded, measured, then tested directly.** Transcription index quantifies the copying; the perturbed-plan experiment **was run** and shows transcription in 11 of 13 local models, binding in 2.
R3.1	"Agentic" is not earned	**Accepted.** Retitled; benchmark 2 supplies a real loop; bounded repair arm since **run** at the perturbed condition (§5.1).
R3.2	Why an LLM at all?	**Accepted, and conceded further than asked.** Introduction concedes the deterministic baseline; Discussion concedes the best lean plan is itself a shell script. Templater baseline **not run**.
R3.3	Scope beyond one workflow	**Accepted in part.** Second workflow class run; it carries no plan gradient and does not extend the plan-sufficiency result.
R3.4	Conventional variant-calling metrics	**Accepted.** Table S12; new Results section.
R3.5	Exploratory vs confirmatory	**Accepted.** The gradient is declared exploratory; benchmark 2's confirmatory status is also withdrawn, for two named reasons.
R3.6	Planner comparison	**Accepted in part.** The documentation-authored arm is answered by v1g. Human-expert and second-frontier planner arms **not run**.
R3.7	Recategorize the error suite	**Accepted, with a worse result.** Two of seven patterns inject no error; true-error recovery is 12.5%, not 41.7%. Expanded suite **not run**.
R3.8	Statistical honesty at n = 3	**Accepted.** Clopper–Pearson intervals throughout; headline raised to n = 10; the discriminating conditions v1g/v1.25/v1.5 also raised to n = 10 (31/126, 63/127, 109/127); the flagged adjective removed.
R3.9	Seed semantics	**Accepted.** Folded into C4.
R3.10	Heatmap n = 2, 3, 6 vs "three times"	**Accepted.** One exclusion rule stated; nothing dropped for a bad answer; Track B pooling explained.
R3.11	Cross-platform divergence	**Accepted in part; premise contested.** The builds are not identical (GGUF q4_K_M vs MLX 4-bit); accuracy claim across platforms withdrawn. Standardised benchmark **not run**.

ID	Issue	Disposition
R3.12	Median outside IQR	**Accepted; correct.** Table regenerated by script with an ordering assertion; the previous table had four of five rows wrong.
R3.13	Honest cost accounting	**Accepted, and it defeats the paper's original motivation.** Full model, break-even, annualised crossover, API-side labour term.
R3.14	Figure numbering broken	**Accepted.** All cross-references audited; a figure-number-to-filename manifest is embedded in Methods.
R3.15	De-promotionalize	**Accepted.** "Winner", "protagonist", "all you need is a plan", "pick the cheapest box" removed; the generational framing was also removed as underpowered.
R3.16	Reproducibility metadata	**Accepted.** Table S8 script-generated with digests, quantisation, engine and tool versions; a parser fault found and reported.

Tier-3 experiments

ID	Experiment	Status
T3.1	Bounded iterative repair arm	**Run**, at the perturbed condition: bind 2 / repair 3 / no repair 8; class 14/39 with repair against 6/39 one-shot. The v1/v1.25 variant remains unrun (§5.1).
T3.2	Planner comparison (expert / second frontier / documentation)	**Documentation arm answered by v1g; the other two not run.**
T3.3	Exploratory/confirmatory separation	**Partly.** Declared exploratory; the frozen protocol was violated and we say how.
T3.4	Second biological benchmark	**Run**, as a case study, without perturbations.
T3.5	Expanded error suite	**Not run as a designed matrix**; existing suite recategorised instead (R3.7).
T3.6	More replicates	**Run.** n raised from 3 to 10 at the headline condition, and later at the three discriminating conditions (273 further cells).
T3.7	Standardised cross-hardware benchmark	**Not run**; the accuracy claim it would have supported is withdrawn instead.
T3.8	Biomni head-to-head	**Not run**; declined with reasons (§5).

3. Two points of disagreement, and one internal tension

3.1 R3.11 — identical weights do not imply identical output

The premise that the same model on different hardware should produce equivalent output is incomplete. Answering it by compliance would have misled the reader. The builds are not the same build of the same weights. Supplement, *Platforms, engines, and reproducibility metadata*: "the CUDA platforms ran GGUF `q4_K_M` weights under ollama, the Apple-silicon machines ran MLX 4-bit builds." Those are different quantisation schemes, different kernels and different batching. Divergence is the expected result, not an anomaly needing explanation.

Two further engine facts affect the results and are now reported. Flash attention was disabled on the Jetson after "22 illegal-memory-access faults in six hours across 5 of 12 models, against zero faults in roughly 41 h after disabling it". The key–value cache was set to `f16` rather than `q8_0`. 400 earlier local runs made under the quantised cache were discarded rather than pooled.

We do not use this to keep a claim we cannot support. The cross-platform accuracy claim is withdrawn: "accuracy was not measured comparably across platforms at these sample sizes, so no claim of platform-independent accuracy is made (Supplementary Table S13)". The standardised fixed-prompt, fixed-length, matched-configuration benchmark (T3.7) that would replace it was not run.

3.2 R2.8 — no published rubric exists for this task shape

We do not accept the implication that an established scoring scheme was available and ignored. The published bioinformatics-agent evaluations score different objects. BixBench grades open-answer responses to curated questions over Dockerised capsules. Biomni applies per-task rubrics across a heterogeneous biomedical task set. Neither scores the object under test here. That object is whether a single generated shell script executes, produces schema-valid outputs, and reproduces a canonical call set. The GA4GH germline benchmarking best practice (Krusche et al. 2019) is the nearest convention, and only for the *variant* half. It also assumes a truth set independent of the caller under test, which this design does not have.

What we did instead is document every departure rather than defend the rubric in prose. Supplement Table S5 lists each scoring choice, the closest published convention, and the departure with its reason. The table's note states: "This replaces the promise of an enumeration that the first version of this manuscript made and did not keep." We added the conventional metrics R3 asked for alongside, not instead. The honest cost of having no standard rubric is stated where it belongs: "**no external anchor was obtained** and no number here sits on a published scale".

3.3 The R2 internal tension: catalogue out, background in

R2 asked for Table 1's model catalogue to leave the main text (R2.2). R2 also asked for more technical LLM background — MoE, KV-cache scaling, the anatomy of an agentic loop — to enter it (R2.3). We read these as one instruction, not two conflicting ones: **explain the concepts a reader needs to size hardware, and delete the list of models that reader will never run**. A catalogue of fourteen model tags is a lookup table with a shelf life of months. The reason a 35 B mixture-of-experts model fits where a 27 B dense model does not outlives every tag in the table.

Resolved as follows. The background went into the Introduction.

> "a mixture-of-experts (MoE) model routes each token to a subset of experts, so per-token compute scales with *active* parameters while residency scales with *total*, and the key–value cache grows with context length rather than model size, more slowly than a naive projection suggests because modern models share key and value projections across heads (Table 1)."

The catalogue went to the Supplement. Main-text Table 1 is now four local models chosen to show the dense/MoE contrast, plus the API executors. Its caption states the demotion.

> "This is a selection, not a catalogue: fourteen local models appear in this study, and the full list with digests, declared context lengths and per-model default decoding parameters is Supplementary Table S8."

Both columns R2 objected to — "coding variant" and "reasoning" — are gone.

One thing did not survive the swap. R2.3 asked for a residency discussion. Residency is exactly where our own numbers do not reconcile. Rather than print a column we cannot defend, Table 1 carries this note.

> "**No residency column is given**, because this study's own residency figures for one of these models do not reconcile; every residency measurement made, with the discrepancy stated inline, is Supplementary Table S4. Residency was measured for four of the fourteen models and not for

the other ten; filling that column costs roughly six minutes of cold load per model, was not done, and no figure is estimated."

---

4. Point-by-point responses

4.1 Consensus items

C1 (= R2.10, R3.1) — Single-pass `run.sh` generation is not an agentic loop: no inspection of intermediate output, no repair, no decision to add steps

The point. The paper called its system agentic. It is not. The model emits one script and never sees what happens to it. There is no observation, no repair, no ability to add a step in response to what it finds. Nothing about the design earns the word.

Response. Correct, and the word is removed from the title. We did three things: stated the limitation in the paper's own voice wherever the design is described, added a second benchmark that actually has the loop, and ran the bounded repair arm (T3.1) this comment asked for.

What changed.

- Title: "agentic analysis orchestration" is gone. The new title names the measured objects — local 4-bit executors, literal command lines, a fixed pipeline.
- Introduction: "Benchmark 1's harness is not an agent harness at all — one prompt in, one script out, executed externally, with no loop, no observation and no repair — which isolates plan detail as the only variable but means the system should not be called agentic, and benchmark 2 was added to supply the missing loop."
- Methods, *Study design*: "Benchmark 1 was single-pass by construction: one prompt in, one bash script out, executed and scored with no opportunity to observe the outcome or retry. Benchmark 2 was multi-turn: the executor issued tool calls against a live public server, read invocation and job state back, and carried state across session boundaries."
- Results, *Injected failures*: the single-pass constraint is turned into the motivation for the loop rather than defended — "These runs had no opportunity to retry, so **a 12.5% recovery rate under a single pass is the strongest internal motivation for the repair loop reviewers asked for**, and the honest reading of benchmark 1 is that it measures what a model does without a loop."
- Results, *Failure modes visible only in the agentic setting*: three modes that a single-pass design cannot produce — an uncompleted final step invisible to an invocation-only scorer, a model stuck on its own stale durable note (losing ~2 h of correct upstream compute), and a zero-tool-call empty turn in 6 of 30 launches.
- Supplement, *Agent harnesses and bioinformatics agents*: "That was a deliberate choice — it isolates plan detail as the only variable — but it is also precisely the limitation three reviewers of the first version of this manuscript identified, and the resulting system should not be called agentic."
- Results, *Three attempts separate transcribers that repair from transcribers that do not*: the bounded repair arm, run at the perturbed condition (main-text Table 7; Supplementary Table S18).

Addressed by new data. The bounded repair arm was run after the second review cycle, at the perturbed condition — the setting where every transcriber fails. Design: on a nonzero exit, the model saw its own script, the exit code and the last 40 lines of the log, and could resubmit, up to three attempts. The retry signal was the exit code only; the score was never shown. Result, in tiers: the two binders were perfect on attempt 1; `qwen3.6:27b` and `qwen3.8:27b` were perfect on attempt 2 in all three seeds; `laguna-xs-2.1` in two of three; the remaining eight models recovered 0 of 24 seeds. Class success with repair is 14/39, against 6/39 one-shot. What this concedes: the single-pass design hid a real capability. For three models, one look at the error did what no plan prose did. What it does not concede: that a loop makes the class work. Eight

models failed all three attempts, with the error naming the missing file and the correct path in their prompt. Full disposition: §5.1.

C2 (= R1.3, R2.4, R3.3, R3 preamble) — Scope is one small workflow; conclusions are broader than the evidence

The point. One 7-step mtDNA workflow on one small dataset cannot support claims about genomic analysis generally.

Response. Accepted. A second workflow class was run, and the claims were narrowed to a boundary we can state precisely — and the boundary turns out to be sharper and more damaging than "one workflow is not enough".

What changed.

- A second workflow class was run: 30-step RNA-seq quantification on usegalaxy.org, reported as a case study (Results, *Case study*; Figure 5; Table S14).
- The scope boundary is now stated three times, because it bounds every claim. Introduction: "**the two classes run here are the two in which the plan can state the answer, and the four not run are ones in which it cannot.** In each, at least one parameter is a scientific judgement made against the data — filtering thresholds, clustering resolution, assembler parameterisation, database version, peak-calling thresholds — so there is no canonical value for a plan to carry and no invocation for an executor to transcribe."
- Abstract: "Both tasks in this study were selected for having no data-dependent parameter choices, so that a plan could state the answer; that criterion bounds every result below."
- Discussion, *Scope*: "**The plan-sufficiency result is conditional on tasks with no data-dependent parameter choices**, and may be in part a restatement of the criterion by which the two tasks were selected. The results support the use of local executors for binding and running pipelines; they say nothing about local executors making analytical decisions."
- The second benchmark is explicitly not allowed to do work it cannot do: "the case study does not extend the plan-sufficiency result, because it contains no plan gradient" and "no claim is supported by pooling the two."

Concession. The near-tautology is now in the paper rather than left for a reviewer to find: the tasks were selected for having no data-dependent parameters, and the headline finding is that a plan works when it states the parameters. We do not think this voids the result. The *quantitative* content — that literal invocations, not prose, account for the difference — is not implied by the selection criterion. But we no longer present the qualitative headline as if it were a discovery independent of the design.

C3 (= R2.11, R3.4) — Jaccard alone is the wrong metric; needs conventional measures and a defined pass/fail

The point. A tolerant Jaccard overlap is not a variant-calling metric anyone uses, and the manuscript never said what counts as success.

Response. Accepted in full. Conventional metrics were added, the pass/fail criterion is now defined in exactly one place, and the resulting section reports something that hurts: precision is 1.000 by construction and therefore uninformative about calling quality.

What changed.

- Methods, *Scoring*: the three-rung ladder is defined once, in the released scorer's own field names — "**M1** (`m1_executes`), `bash run.sh` returned 0 within 600 s; **M2** (`m2_schema`), every expected output exists and `bcftools` accepts every VCF; **M3** (`m3_jaccard`), the per-sample variant output matches the answer key." Two properties that govern the whole Results are stated with it: M3 is scored 0 whenever

the script exits non-zero, and M3 "**is the tolerant Jaccard overlap** ... not a per-sample success fraction".
- New Results section, *The conventional metrics are not degenerate*: TP = 1,042, FP = 0, FN = 1,874 over 324 reasoning-disabled runs, with per-condition recall (0.086 at Track B, 0.306 at v1, 0.485 at v1.25, 0.886 at v1.5) reported against the run-level success rate in each (Table S12).
- Table S5 enumerates each departure from the nearest published convention.

The concession this produced. "precision is 1.000 by construction, because the truth set is the canonical output of the same deterministic caller." And: "What they cannot do is separate a good caller from a bad one, because the truth set is the same caller's own output, and no perturbation series that would vary accuracy continuously was run." The metrics measure plumbing, not calling. We report them because they *do* separate two failure classes the run-level metric conflates — 8% of runs get partial biology right, and 2.5% get the biology entirely right and then exit non-zero — but they are not a variant-calling evaluation and we no longer imply that they are.

C4 (= R2.6, R3.9) — "Seed" does not control LLM determinism, and API seed semantics differ from local

The point. Reporting an integer "seed" implies reproducible sampling that does not exist, and the API runs do not take a seed at all.

Response. Accepted, and the audit prompted by it found a worse problem than the one raised.

What changed.

- Supplement, *Inference settings*: "For local models an integer seed was passed to the sampler, so at fixed decoding parameters it does select a trajectory through the distribution those parameters define. For the Anthropic API runs the harness constructs no seed argument at all: the integer recorded in those rows is a bare run identifier and is reported as such. Neither case gives bitwise determinism, because batched GPU inference does not guarantee identical results even at a fixed seed. All claims of reproducible sampling have been removed."
- Full decoding parameters are reported: ollama 0.32.5, `temperature = 0.2`, `num_ctx = num_predict = 16384`, with per-model defaults in Table S8.
- The new finding, disclosed in Methods rather than buried: "**A confound, not merely a disclosure:** the harness overrode only `temperature`, leaving `top_k` and `top_p` at each model's own defaults, which are not uniform across families and are absent altogether for `qwen3.8:27b` (Supplementary Table S8), so every cross-model comparison here compares models decoded from different distributions. The gradient itself is a within-model comparison and is unaffected."

See R2.6 for the unrun pinned-decoder re-run.

C5 (= R1.2, R2.9, R3.1) — Missing related work; no baseline comparison

The point. The planner–executor split is prior art and was presented as if new; there is no engagement with Biomni or with end-to-end bioinformatics agents; no baseline.

Response. Accepted. The originality claim is withdrawn, the literature is cited, and the absence of an external anchor is now stated as a defect rather than left implicit.

What changed.

- Introduction: "Separating a specification from a process that carries it out is not a contribution of this paper. In genomics it is the design of every workflow system in routine use ... The same split runs through the language-model literature, from plan-and-solve prompting and least-to-most decomposition to ReAct, HuggingGPT, compiler-style graph dispatch and Reflexion (Wang et al. 2023; Zhou et al. 2023;

Yao et al. 2023; Shen et al. 2023; Kim et al. 2024; Shinn et al. 2023) … We adopt this pattern; we do not propose it."
- Supplement, *Agent harnesses and bioinformatics agents*: Biomni, BixBench, BioMaster and BioAgents are each described with what they score.
- The anchor problem is stated plainly: "**no external anchor was obtained**: none of the numbers reported here can be placed on a published scale, so a reader cannot tell from this paper whether 34 of 36 perfect runs at the detailed plan is good by any external standard." And the cheap fix is named: "BixBench ships 53 public Dockerised capsules precisely so that new executors can be placed on a common scale, and running this study's executors on a subset of them is a bounded experiment on public artifacts that would convert this declination into an anchor. It was not run, and this is the sharpest reproducibility gap in the paper's positioning."
- Two external comparators for benchmark 2 are used with their protocols stated: a human manual execution on the same server, and eight frontier models acting as both planner and executor on the same data at $2.82–$131.83 per run (Nekrutenko 2026), which we mark as "by the present author, with unpublished plans and no replicates, which bounds API cost and is not a frontier control."

Not addressed by new data. The Biomni head-to-head (T3.8) was not run. See §5.7.

4.2 Reviewer 1

R1.1 — Security guardrails are absent

The point. The paper describes a language model writing shell code that is then executed, and says nothing about what constrains it. This was R1's first comment.

Response. Accepted. The honest answer is that benchmark 1 ran essentially unsandboxed, and we say so rather than describing an intended architecture. We also replaced the assertion that nothing bad happened with a measurement.

What changed.

- Methods: "Generated scripts were executed on the model host in a per-cell working directory under a pinned conda environment, killed at 600 s, without human review and with no container, VM, seccomp profile, namespace, cgroup quota or egress restriction. Only the wall-clock kill, the PATH-restricted inventory and the working directory are enforced; the constraints on absolute paths, package managers and network fetches are prompt instructions a model is free to violate … **The local arm does not meet the sandboxing bar a production deployment needs and is not a deployment template**".
- Results: "A lexical audit of the 766 archived `run.sh` files replaces the claim that no destructive or escape behaviour was observed: none invokes a package manager, network fetch tool or `sudo`, none directs output to an absolute path outside `/dev/` and `/tmp/`, and 17 (2.2%) write under `/tmp/`. One script — `rm -f /tmp/*.txt /tmp/temp_*.vcf.gz` — **is a genuinely out-of-sandbox action**, and it came from a no-plan run; the `/tmp/` writes concentrate there generally, 11 of 303 no-plan scripts against 6 of 463 written with a plan, because a model given no plan invents its own scratch-space convention and the convention it invents leaves the sandbox."
- A first-version claim is withdrawn by name: "no audit of available escalation paths on the JetPack image — sudoers entries, docker group membership, writable setuid binaries on `PATH` — was performed, so the first version's claim of 'no route to elevation' is withdrawn."
- Another is corrected: the benchmark-2 isolation claim "the blast radius of any error is one history" "was wrong" — a Galaxy API key is account-scoped, and what narrowed the blast radius was the MCP server's bounded verb set, not the credential.
- Prompt-injection exposure, previously unmentioned: "Attacker- or accident-controlled text reached the executor through at least five channels here: dataset and history names on a shared multi-user public

server, job stderr, tool output read back through MCP, the descriptions of the MCP tools themselves, and — in benchmark 1 — 200 lines of adversarially shaped stderr injected deliberately through a PATH shim … no indirect-prompt-injection test was run."
- Supplement states the production bar: sandboxing "is a prerequisite for production use of this pattern. The local arm of this study does not meet that bar and is not a deployment template."

R1.2 — Cite the planner–executor pattern properly and drop originality language

The point. Plan-and-solve, least-to-most, ReAct, hierarchical decomposition and LLM-compiler approaches are established. The architecture is not the contribution.

Response. Accepted without reservation.

What changed. Introduction now carries the citations (Wang et al. 2023; Zhou et al. 2023; Yao et al. 2023; Shen et al. 2023; Kim et al. 2024; Shinn et al. 2023), the genomics workflow-system lineage (Di Tommaso et al. 2017; Mölder et al. 2021; Amstutz et al. 2016; Galaxy Community 2024), and the bioinformatics survey (Alam and Roy 2025). The contribution is restated narrowly: "The contribution claimed is narrow: a measurement of that arrangement on genomic work, on hardware a laboratory can buy outright, with plan detail as the independent variable."

R1.3 — Generalization to other workflow classes and to production

The point. How does the split translate to other analyses, and what changes on a shared cluster — scheduling, queueing, provenance, failure paging, multi-user isolation?

Response. Accepted, and answered partly with data rather than only with prose, because usegalaxy.org *is* a production setting.

What changed.

- Supplement, *Production concerns for a shared-cluster deployment*: "The case study supplies part of the answer to what changes in a production setting, because usegalaxy.org is one. The executor operated under an external scheduler with real queueing, per-tool containerisation and server-side provenance recorded independently of the agent."
- Table S20 lists four untested production concerns — executor process isolation, credential scoping, queue-time behaviour, unattended failure notification — each with what benchmark 2 supplied, what remains untested, and a recommended mitigation. The caption states "No row is a tested result."
- The failure-notification row is a concrete observed failure, not a hypothetical: "six counting jobs aborted approximately two hours into an unattended run, with no notification path, and were noticed by a human."
- Discussion derives three deployment recommendations from observed failure modes only: bound the tool set's *inputs* as well as its verbs; treat durable agent memory as a cache that must be invalidated; require a dataset or job identifier beside every number an agent reports.
- Workflow-class generalization is Table S19 (see R2.4).

R1.4 — Data governance at the planning step, and planner instruction detail

The point. A commercial planner must not receive proprietary configs, local layouts, sample identifiers or system metadata. Separately, the manuscript must describe how the planner is prompted, by category, including whether recipes carry safety/validation checks; pointing at GitHub is not enough.

Response. Accepted on both halves. The governance half produced a finding against us, which we report rather than soften.

What changed — governance.

- The boundary is drawn in Figure 1 and immediately qualified: "The dotted line marks the data-governance boundary as it was used here; it is not enforced by the harness, and the frontier executor arms of benchmark 1 cross it, because their prompts carry the dataset manifest."
- A payload audit was run: "`revision/scripts/payload_audit.py` reconstructs every outbound payload and scans it against a deny-list of local-identity tokens (Supplementary Table S3)".
- The result, in Methods: "The split also defines a data-governance boundary, and **it was respected by the planner role only**: the frontier *executor* arms — 315 API cells in all — transmitted the executor prompt, which carries the dataset manifest with this study's own subject labels."
- Discussion states the consequence at full strength: "no host name, conda prefix or credential appeared in any outbound payload, but every executor payload carried the operator's home directory — exported by the very constraint line that prohibits it — and all three planner meta-prompts carried the study's sample identifiers, the class a data use agreement is most likely to restrict. **The sharpest illustration is this study's own design: the frontier-executor arms, whose prompts carried subject labels for a mother–child pair to a commercial API, would not have been permissible under a controlled-access DUA.**"
- The positive argument R1 identified is now made explicitly, in *What the split buys once cost is off the table*: with cost off the table, the split buys an abstraction boundary that "under a dbGaP or EGA data use agreement, an IRB transfer restriction or HIPAA is the difference between a controlled disclosure and a data transfer."
- One gap is stated: "benchmark 2's MCP traffic is neither archived nor reconstructable and is outside the audit."

What changed — planner instruction detail.

- Supplement, *Prompt structure for the planner and the executor*, categorises the three planner meta-prompts on five axes — role and audience framing, whether bash code is permitted, whether literal flag values are required, the step-count limit, and whether validation and retry prose is requested — then does the same for the system prompt (seven hard constraints, three of which are harness-enforced) and the five-slot user template.
- Safety and validation checks are located exactly: "Safety and validation checks are carried by exactly one plan condition. `v2_defensive` adds a `try()` helper, per-step validation, retry-once semantics, a `failures.log`, and an exit-code policy. The other seven plan conditions carry none of these."
- The files are reproduced rather than referenced: "The first version of this manuscript stated that these files were reproduced below the tables and did not reproduce them. They are reproduced here." Three planner meta-prompts, the system prompt, three user templates and plan files v1 and v2 are printed verbatim in the Supplement.
- A disclosure that materially affects the gradient's interpretation was found while doing this: "Readers checking the transcription argument should note that `PLANNER_PROMPT.md` — the meta-prompt behind the *lean* condition — itself contains the literal read-group string and the `lofreq call-parallel` subcommand name." Main text states the consequence: "**the gradient is in part a gradient in how much literal syntax was demanded of the planner, not purely in model-authored detail.**"

Remaining gap. The Galaxy plan files, the fabrication study's code and per-cell outcomes, the defect ledger with its diffs, and the `loom` session logs are not released. This is stated in Methods, in Limitations, and at every point of use, with the source marked as "a hand-typed contemporaneous record of live API read-back — **the same evidentiary class as an assertion in this manuscript**".

4.3 Reviewer 2

R2.1 — Reads as an implementation guide, not a paper

The point. The LLM is the subject and genomics is the substrate. The introduction should start from the genomics problem — routine per-sample analysis at scale in a small lab and the human cost of babysitting it — and introduce LLM orchestration only afterwards as one candidate solution.

Response. Accepted. The Introduction was rewritten in that order, and the motivating claim R2's framing invites — that this saves human time — is explicitly *not* asserted, because we did not measure it.

What changed.

- Introduction now opens on the laboratory, not the model: "A laboratory that generates its own sequencing data does the same analysis many times. Each batch goes through a pipeline settled months ago and not in question, and none of that work is free: someone has to bind the pipeline to this batch's file names, run it, notice which samples failed, and re-run them."
- The question is posed as a genomics question with the model as one candidate answer, and the unsupported claim is disclaimed in the same paragraph: "Human attention is the motivation for this work and this study did not measure it, so the claim that this architecture saves human time is conditional on unsupervised operation, which this study did not achieve, and we do not assert it."
- The alternative solutions come *before* the LLM, not after: "For any pipeline that is stable and repeatedly run, a validated shell script under version control, a Snakemake or Nextflow rule set, a CWL description, or a saved Galaxy workflow is simply better than a language model ... deterministic, auditable, free to invoke, and no GPU."
- The results are stated as measurements about a class of executor rather than as guidance: " 'Class' and not 'identity': the local arm is 4-bit quantised, at most 35 B, open-weight and served by ollama locally, while the frontier arm is unquantised, of undisclosed size, from a single vendor, served over an API and decoded under different sampling parameters. Four variables move together and this design separates none of them."

Addressed by new data, and the objection is confirmed for most models. The anti-copy control implied by this framing — a perturbed v2 plan — **was run** after the second review cycle, in one variant: plan v2 verbatim, the reference moved to `data/ref/GRCh38_chrM/rCRS.fa`, the real path stated in the prompt's DATASET section, and the sandbox validated with a hand-written copier (fails) and binder (succeeds) before any model ran. Thirteen local models × 3 seeds gave **6/39 perfect against a 34/36 unperturbed control, and every model is 3/3 or 0/3**: eleven of thirteen — including the original headline model `qwen3.6:27b` — wrote the plan's stale path verbatim into their scripts and died at `bwa index`, and two (`gemma4:31b`, `gpt-oss:20b`) rebound every path and produced the correct call set in all three seeds. R2's objection is therefore confirmed as a measurement for 11 of 13 local models, and the manuscript now carries it as the central result (Results, *A one-path perturbation separates transcription from binding*; Table 6; Supplementary Table S17). The title and abstract were rewritten around it. Its limits are stated in Limitations: three seeds per model, a single perturbation type (one moved path), and no frontier arm. See §5.3.

R2.2 — Move Table 1 to the supplement; drop the "coding variant" and "reasoning" columns

The point. Table 1 is a model catalogue, and two of its columns are meaningless — all current models reason, and this is not a coding benchmark. Keep a small main-text table with only the models evaluated and the parameters that matter for hardware fit.

Response. Accepted exactly as specified.

What changed. Main-text Table 1 is now six rows: four local models plus the two API groups, with columns Architecture, Total params, Active params/token, Quantisation and Role. The "coding variant" and "reasoning" columns are deleted. The caption states the demotion and the reason: "This is a selection, not a catalogue: fourteen local models appear in this study, and the full list with digests, declared context lengths and per-model default decoding parameters is Supplementary Table S8." The quantisation non-uniformity

that the old catalogue hid is now on the face of the table: "quantisation is not uniform, and `gpt-oss:20b` ships MXFP4 while every other local model is `q4_K_M`."

R2.3 — Add the conceptual background: MoE vs dense and VRAM residency, KV-cache scaling, the anatomy of an agentic loop

The point. The main text should teach the concepts a reader needs to size hardware and to understand what an agent loop is, in place of the catalogue.

Response. Accepted; see §3.3 for how this was reconciled with R2.2.

What changed.

- MoE and KV cache, Introduction: "a mixture-of-experts (MoE) model routes each token to a subset of experts, so per-token compute scales with *active* parameters while residency scales with *total*, and the key–value cache grows with context length rather than model size, more slowly than a naive projection suggests because modern models share key and value projections across heads (Table 1)."
- Agent-loop anatomy, Supplement: "software that presents tools to a model, runs the resulting tool calls, feeds the outputs back, and decides when to stop", with the Introduction stating what benchmark 1's harness lacks against that definition.
- Table 1 instantiates the dense/MoE contrast concretely: `qwen3.6:35b-a3b` is 35 B total at 3 B active per token, against `qwen3.6:27b` at 27 B/27 B.
- Residency is deliberately not tabulated; see §3.3. Table S4 reports every residency measurement made with the discrepancy stated inline.

R2.4 — Task taxonomy: enumerate candidate workflow classes with plan complexity and I/O footprint

The point. Even though only one class was run, a table of candidate classes sets scope honestly and shows the framework generalizes in principle.

Response. Accepted, but the table was moved out of the main text and its qualitative rows marked, because in the first version four of six rows were author opinion holding numbered-table status.

What changed.

- Table S19 gives six classes — alignment/variant calling, RNA-seq quantification and DE, single-cell RNA-seq, de novo assembly, metagenomic profiling, ChIP-seq/ATAC-seq peak calling — with representative analysis, step count, plan complexity, I/O footprint and whether it was run.
- Provenance is marked per cell: "Step counts and I/O footprints are measured only for the two classes that were run. For the four classes that were not run, plan complexity and I/O footprint are qualitative estimates by the author with no measurement behind them, and they are marked as such; no numeric step count is offered for them".
- The reason for demotion is stated: "four of its six rows were author opinion with no measurement behind them and a numbered main-text table gave them a standing they had not earned."
- The taxonomy's most important output is promoted into the main text as prose, because it is the paper's scope boundary rather than a survey: the two classes run are the two in which a plan can state the answer (see C2).

R2.5 — Survey of open-source harnesses, early in the draft

The point. Claude Code, OpenCode, Aider, OpenHands, Goose, Biomni and others exist; the paper should say what each does that this harness does not. This also makes the single-pass design an explicit choice rather than an apparent oversight.

Response. Accepted in substance. We kept the survey and deleted the table that carried it, because the table could not be checked.

What changed.

- Introduction: "The execution side is supplied in practice by an *agent harness*; no comparison among the several in wide use by mid-2026 was run here, benchmark 2 used `galaxyproject/loom` for deployment reasons given in the Supplement, and nothing in the results is known to be harness-independent."
- Supplement names the harnesses — "Claude Code, OpenCode, Aider, OpenHands, Goose, Cline and Continue among them" — and explains the deletion: "The first version of this manuscript carried a capability table for these harnesses; it is removed, because every row answered yes to every capability column and no reader could check any cell against a pinned release."
- The `loom` choice is given as deployment constraint, not judgement: "it runs headless with no terminal attached, and it keeps a file-backed `notebook.md` that survives session boundaries, which the multi-hour Galaxy polling loop requires."
- The harness confound is stated where it costs us: two of the three benchmark-2 failure modes "are properties of `loom` as much as of the executors".
- The design choice is made explicit exactly as R2 intended, in the Supplement passage quoted under C1.

R2.6 — "Seed" does not control determinism; report the sampling parameters

The point. Determinism in LLM inference is set by temperature, top-p, top-k, repeat penalty and max tokens, not by a seed integer.

Response. Accepted; see C4 for the rewrite. The audit this triggered found that the harness never controlled `top_k` or `top_p` at all, which is a confound in every cross-model comparison in the paper. We report it as such.

What changed. Supplement, *Inference settings*: "`top_k` is 20 for the Qwen 3.5 and 3.6 models, 64 for Gemma 4, unset elsewhere; `top_p` is 0.95 for those Qwen models, Gemma 4 and GLM, 1 for Nemotron, unset elsewhere. The family label is not a safe shorthand: `qwen3.8:27b`, the benchmark-2 dense arm, declares neither ... **This is a confound and not merely a disclosure**: every cross-model comparison in this paper — the gradient, the repeated-sampling table, the abstract's contrast between arms — compares models decoded from different distributions, and the model that fails at the headline condition (`nemotron-3-nano:4b`, 1/10) and the models that pass are not decoded alike."

The within-model gradient is what the paper's central claim rests on. It is unaffected, "because each model's decoder is constant across its own columns". We say that rather than letting the disclosure appear to void everything.

Not addressed by new data. The pinned re-run was **not done**. The cost is quantified: "re-running the headline cell (12 models × 10 seeds) and the discriminating cell (12 models × 3 seeds) with `top_p` and `top_k` pinned is 156 cells, roughly 4 GPU-hours against the 75.27 h already spent ... **That was not done in this revision.** Until it is, the ordering of models under a pinned decoder is unknown, and any cross-model ordering claimed here must be read as provisional." See §5.4.

R2.7 — Promote error handling from Methods to Results, with the injection mechanism and how recovery was scored

The point. Error handling was buried in Methods and under-described.

Response. Accepted. It is now a Results section, and rescoring it as R3.7 required made the result substantially worse for the paper.

What changed.

- Mechanism, Methods: "Tool failures were injected with PATH shims applied only to the targeted tool, so the generated script saw nothing but ordinary exit codes, stderr and output files. Each executor–plan combination comprises 39 cells: 8 pattern–target combinations injecting a true error × 3 seeds, 4 injecting no error × 3 seeds, and 3 uninjected controls."
- Results, *Injected failures*: the 24 true-error cells split by modal handle category into crash in 15, forward the error in 6, and recover in 3. That is **a recovery rate of 12.5% (exact 95% CI [0.03, 0.32]) against a crash rate of 62.5% ([0.41, 0.81])**, with a mean M3 of 0.000. The section states: "Nothing here shows that any executor handles injected failures well."
- The pooled figures are explicitly disqualified: "the pooled figures of 0.417 by handle category and 0.750 by the per-sample recovery flag are not quotable as recovery rates."
- A second result is reinterpreted downward: the local and frontier executors agreeing in every one of 39 cells "is a degeneracy, not a parity finding ... the handle category at the baseline plan is essentially determined by the injected pattern", with two bounds on that reading — the per-cell artifacts were not archived, and the frontier comparator is a different model generation.
- The model term is not zero: under v2_defensive, `qwen3.6:27b` and `claude-opus-4-7` reach recover 9 / partial 15, `qwen3.6:35b-a3b` reaches recover 3 / partial 21 with mean M3 *falling* to 0.208, and `granite4` crashes in 24 of 24 cells — "a defensive plan changes error handling and how much it helps is model-dependent."
- Discussion draws the usable consequence: "a plan must be validated against the executor that will run it, because a defensive plan helped two executors, left a third worse than before, and destroyed a fourth."

R2.8 — Align scoring with published practice; cite prior bioinformatics-agent scoring schemes and justify departures

The point. The rubric is custom. Map it onto established measures where possible and justify each departure.

Response. Accepted on the substance — every departure is now enumerated — while contesting the premise that a published rubric existed for this task shape and was ignored. See §3.2.

What changed. Supplement Table S5, *Scoring, and how it departs from published practice*, gives one row per scoring choice with the closest published convention (BixBench final-answer scoring, Biomni task rubrics, GA4GH germline benchmarking best practice per Krusche et al. 2019) and the departure with its reason. Conventional variant-calling metrics were added alongside the custom ladder, not as a replacement (Table S12). The absence of an external anchor is stated as a defect of this paper's positioning rather than as a property of the field.

R2.9 — Related work must engage Biomni explicitly, plus end-to-end GWAS and variant-effect agent work

The point. The nearest work in the field is not cited or positioned.

Response. Accepted for positioning; the head-to-head is declined with reasons.

What changed. Supplement, *Agent harnesses and bioinformatics agents*, characterises each system by what it actually scores: "Biomni couples a large tool and database inventory to a frontier model and scores it across a wide range of biomedical tasks, including ones requiring literature retrieval and experimental design (Huang et al. 2025); BixBench grades agents on 53 Dockerised computational-biology capsules with 296 curated open-answer questions (Mitchener et al. 2025); BioMaster wraps a plan/task/debug/check loop around retrieval-augmented generation for RNA-seq, ChIP-seq, scRNA-seq and Hi-C (Su et al. 2025); and BioAgents fine-tunes small open-weight models for local execution (Mehandru et al. 2025)."

A comparison table from the first version was deleted. The reason given: "every row of it explained why the numbers are not comparable, which is an apology rather than an anchor, and it is removed".

Not addressed by new data. See §5.7 for the declination and what a head-to-head would take.

R2.10 — Single-pass generation is not agentic

Identical in substance to C1 and R3.1; see C1 for the full disposition, including the repair arm. In addition, the Discussion ties the paper's central claim to the single-pass constraint rather than to the models, and now reports the arm that tested it.

> "**The requirement for near-executable plans holds only under this study's single-pass constraint, which is a property of the harness and not of the models**. A bounded repair arm was run at the perturbed condition, with the error fed back and up to three attempts. It answers the question in part. Three of the eleven transcribers fixed the stale path when shown the error, and eight did not."

R2.11 — Jaccard is degenerate at this scale; state the pass/fail criterion in one place

The point. With ~5 truth variants, Jaccard takes a handful of discrete values. Report the underlying counts or a per-variant confusion matrix, and say once and explicitly what a successful run is.

Response. Accepted, and the degeneracy is worse than "a handful of values" — on the local arm it is binary.

What changed.

- Table 3 is reported as counts, not means, with the reason in the caption: "**The local arm is reported as a count of runs producing the correct call set and not as a mean score**: on all 324 local reasoning-off cells M3 takes only the values 0.0 and 1.0, so a local 'mean M3' column is the perfect-run proportion written twice and invites the reader to believe partial biological credit was measured, which on this arm it was not."
- The metric is retained only where it is not degenerate: "The tolerant Jaccard is retained only for the frontier arm, where it is non-degenerate; its cells there take the values 0.0, 0.833, 0.938 and 1.0, so a Track-B mean of 0.825 coexists with only 13 of 27 runs perfect."
- Pass/fail is defined once, in Methods, in the released scorer's field names (see C3).
- Underlying counts are reported: TP/FP/FN totals, per-condition recall, and the two mechanisms that produce the gap between the curve without the exit-status requirement and the curve with it — "**26 of the 324 runs produced exactly 2 of the 9 truth variants**" and "**8 of the 324 runs wrote all four VCFs and then exited non-zero**, of which 6 had recovered all nine truth variants".
- The exit-status requirement is named as a choice, not a fact: "Collapsing such a run to 0 is defensible — a pipeline that exits non-zero has not finished — but it is a choice, and it is what makes the run-level metric degenerate, not the biology."
- No pooled F1 is reported, with the reason stated: "it averages over conditions that share no denominator of interest and re-expresses a binary."

4.4 Reviewer 3

R3.preamble — Success at the detailed end may be transcription of near-executable syntax rather than orchestration, and the design cannot distinguish the two

The point. The paper's headline result may be an artifact of asking a planner for copy-pasteable commands and then rewarding models for pasting them.

Response. We concede this before the first result rather than defending against it, and we measured its size.

What changed.

- Introduction, before any result: "at the detailed end of the gradient the plan was *specified* to be copy-pasteable, because the v2 meta-prompt instructs the planner to emit 'every flag, with its exact value' on the stated assumption that 'the implementer will copy-paste your invocations into a shell script'. A model that succeeds at v2 is therefore doing at least partly what the condition was designed to require, and we measure how much of it is transcription rather than conceding the point, twice: indirectly, by a token-recall index ... and directly, by executing the same plan against inputs its literal paths no longer match."
- New Results section, *How much of the plan is copied*, defines a transcription index over 514 archived scripts and reports: correct no-plan pipelines already contain 0.505 of v2's literal command tokens; plan-conditioned correct scripts at v2 contain 0.749. "**about half of v2's literal command tokens are what any correct pipeline for this task contains regardless of plan, and roughly a further quarter is transcription.**"
- A claim from the first version is withdrawn by name: "**the claim that the index tracks the score is withdrawn**: there is no within-condition signal — at v2 successful scripts recover 0.749 against 0.722 for failed ones, and at v1 0.509 against 0.517, the wrong direction."
- The gradient section ends by conceding: "This result concedes the transcription objection rather than answering it. The claim that survives is the one a reader can use: **a small local model reliably produces a working pipeline when it is handed invocations it can copy verbatim, and the explanation around them adds nothing detectable at these sample sizes.**"
- The discriminating experiment **was then run**: a new Results section, *A one-path perturbation separates transcription from binding*, reports plan v2 verbatim against a moved reference path stated in the prompt. 6/39 perfect against a 34/36 control, bimodal by model: eleven of thirteen local models copied the stale path verbatim and failed at the first step, two rebound every path and stayed perfect. "For eleven of thirteen local models, v2 success is transcription ... binding is demonstrable, not hypothetical — and it is a property of individual models, not of the class."
- Discussion now states the resolved form: for most local models plan sufficiency means literal *and current* — "a stale detailed plan is therefore worse than a lean one for a transcriber, since it fails at the first path while looking like the condition that scores 34/36" — and what remains undemonstrated is binding as a property of the class, under other perturbation types, and in frontier executors, none of which the single path-move variant tests.

R3.1 — Fix "agentic" in the title and throughout, or earn it

Accepted; see C1. The title no longer contains the word. The bounded repair arm has since been run at the perturbed condition (§5.1). It adds a bounded retry, not an open agentic loop, and the title still does not claim the word.

R3.2 — Answer "why an LLM at all?" head-on

The point. A lab can save the validated script, parameterize it, or write it in Snakemake/Nextflow/Galaxy. Where does the LLM earn its cost, and where does it not?

Response. Accepted, and conceded further than the reviewer asked, because our own data supports the objection more strongly than the reviewer stated it.

What changed.

- Introduction concedes the baseline first: "The honest baseline deserves stating without hedging. For any pipeline that is stable and repeatedly run, a validated shell script under version control, a Snakemake or Nextflow rule set, a CWL description, or a saved Galaxy workflow is simply better than a language model ... A language model earns its keep at the edges of that template — one-off analyses, adaptation when layouts or tool versions change, a natural-language interface for people who can evaluate a result but do not write code, and *binding*".
- Discussion turns our own best result against the architecture: "The best-performing lean plan, v1.5, is 159 words and ten literal command lines, and local scripts reproduce 0.956 of its tokens. **A 159-word artifact that is ten command lines, version-controlled, reviewed and reproduced at 96% fidelity is a shell script**".
- The remaining question was narrowed by the moved-path experiment: eleven of thirteen models behaved like a fixed stale template, whereas two rebound the path. The deterministic templater itself was not run, so this is evidence about model behaviour under one perturbation rather than a direct model-versus-templater comparison.
- The Conclusion carries the same concession rather than dropping it after the Discussion: "the leanest plan that works here is short and literal enough to be a shell script, a comparison this study did not run."

Not addressed by new data. The deterministic ~20-line templater baseline was **not run**. See §5.5.

R3.3 — Scope beyond a single workflow

The point. One workflow cannot support the paper's conclusions.

Response. Accepted; a second workflow class was run. We are explicit that it does not do everything a second workflow could have done.

What changed. Benchmark 2 varies four things at once relative to benchmark 1 and we say so: "a second workflow class, a shared production server, a 30-step workflow, and an external cost comparator". It reproduced the published quantification — 5,594 genes, 93.1–93.6% assigned of uniquely mapped fragments against a published ~92% — "verified by downloading and counting the counts table rather than by accepting a reported figure".

What it does not do is stated in the same breath. The text says "the case study does not extend the plan-sufficiency result, because it contains no plan gradient". It also says "Plan sufficiency was established during this exercise rather than tested by it". No data-perturbation series was run on this benchmark (Discussion, *Limitations*). The perturbed-plan pass belongs to benchmark 1. See §5.6 for the plan gradient inside benchmark 2, which was not run.

R3.4 — Add conventional variant-calling metrics: precision, recall, F1, TP/FP/FN, allele-frequency error

The point. Report the metrics the field uses, against the ground truth, and keep Jaccard only as a secondary summary.

Response. Accepted; see C3. The result is informative in a way that does not flatter the design.

What changed. Table S12 and a dedicated Results section. Headline: "Over all 324 reasoning-disabled runs, TP = 1,042, FP = 0 and FN = 1,874, and precision is 1.000 by construction, because the truth set is the canonical output of the same deterministic caller. **Recall, however, does not equal the run-level success rate** in any condition except v2: it is 0.086 against 6/108 at Track B, 0.306 against 9/36 at v1, 0.485 against 15/36 at v1.25 and 0.886 against 30/36 at v1.5 (Supplementary Table S12); pooled, recall is 0.357 against a success rate of 0.321."

The limit is stated. Because precision is 1.000 by construction and no perturbation series was run, "What they cannot do is separate a good caller from a bad one". Allele-frequency error enters through the ±0.02 tolerance in M3 rather than as a separate AF-error panel. Table S5 states the reason a scatter would show nothing.

> "The caller is deterministic and the reference is 16,569 bp: a correctly wired pipeline reproduces the answer key exactly, so stratification would have nothing to separate. 0 of 324 runs produced any allele frequency outside the window."

A called-versus-truth AF plot would therefore be a diagonal by construction, and we did not include one.

R3.5 — Exploratory vs confirmatory separation; v2 was written after seeing v1 fail

The point. The gradient is partly fitted to observed failure modes, so it cannot be treated as confirmatory. Freeze the protocol and run it on a held-out workflow.

Response. Accepted. We declare the gradient exploratory, and we also withdraw the confirmatory status of the second workflow, for two reasons we state rather than let a reader discover.

What changed.

- Introduction: "The plan gradient is exploratory, because the most detailed plan condition was written after observing how the leaner conditions failed; benchmark 2 is a case study, and no claim is supported by pooling the two."
- The freezing attempt and its violation are reported in full, in Methods: "The executor-selection rule, stated verbatim because it is invoked repeatedly: *take every local model that scored 3/3 at plan v2 Track A with reasoning off in the benchmark-1 gradient, and from that set take the largest dense model and the largest mixture-of-experts model by total parameter count.* It returns `qwen3.6:27b` and `qwen3.6:35b-a3b`; the dense arm actually run was `qwen3.8:27b`, substituted at matched parameter count after the rule was fixed, so **that arm is exploratory rather than confirmatory.** Plan sufficiency was never confirmatory on either benchmark, because the Galaxy plan was corrected repeatedly while benchmark 2 ran."
- Results, *Case study*: "**Repeated human correction of the plan, and not any change of model, is what produced a clean run.**"

The minimum viable fix R3 proposed — freeze the protocol, run it on a held-out workflow — was attempted and not completed. We report the attempt and its failure rather than presenting benchmark 2 as the confirmatory arm it was intended to be.

R3.6 — Planner comparison: expert-written, second frontier model, documentation-assembled, at matched detail

The point. One frontier planner authored nearly every plan. Without a planner comparison, the result may be about that planner rather than about the presence of executable syntax.

Response. Accepted in part. The documentation-authored arm exists and answers the sharpest form of the question. The other two arms were not run.

What changed.

- v1g is a mechanically extracted plan with a non-model author: "The exception to model authorship is v1g, whose `lofreq` block was extracted mechanically from the Galaxy IUC tool registry (tools-iuc commit `39e7456`)."
- Its behaviour tracks its content, not its authorship: "v1g is **not** an arm of a planner comparison, which would have to hold syntactic density fixed: it is a mechanically extracted plan at v1-level density with a

non-model author, which behaves as its *content* predicts rather than as its authorship or length would, and that is useful evidence that the gradient is not an artifact of one planner's house style."

- The mechanism it reveals is one of the paper's central results: "v1.25 and v1g each add exactly one literal `lofreq` invocation to v1, and they do opposite things: at ten seeds v1.25 lifts the local executors to 0.496 while v1g leaves them at 0.246, indistinguishable from v1 itself. The difference is *which* line ... **The finding is not that more command lines are better. It is that a plan helps when it supplies an invocation the executor can copy verbatim.**" v1g carries *more* information than v1.25 — it even states the one detail models most often get wrong — and performs half as well; the variable that moves the score is whether the command can be pasted and run.
- The seed extension raised the comparison to as many as ten seeds per model over thirteen executors, giving pooled descriptive totals of 31/126 for v1g and 63/127 for v1.25. Because seeds are clustered within models, the primary analysis uses the 13 paired model-level proportions: mean within-model difference 0.253, exact two-sided sign-flip p = 0.027 (Supplementary Table S16).
- Table 4 reports syntax density against outcome on two machines and at both seed counts, with the operational definition of a "literal command line" and its sensitivity disclosed in the caption.
- A rank correlation is deliberately not quoted: "No rank correlation is quoted over five points, two of them tied in both variables; the v1.25-versus-v1g contrast is the argument, and since the seed extension it is a measured contrast ... rather than a two-cell reading." (`SYNTAX_DENSITY.md` reports Spearman +0.85 for command-line count and +0.10 for word count; we regard five points with ties as too few to publish a coefficient from, and give the contrast instead.)

Not addressed by new data. The human-expert and second-frontier planner arms were **not run**; Limitations states "the planner comparison at matched syntactic density was not run, since v1g's density is not matched and the human-expert and second-frontier-planner arms remain unrun". See §5.2.

R3.7 — Recategorize the error suite: a tool that sleeps then succeeds is not an error

The point. Some injected "errors" are latency or log-tolerance tests, not failures. Relabel along true-failure vs noise-tolerance axes and rescore.

Response. Accepted, and it cost the paper its most favourable error-handling number.

What changed.

- Methods: "**Two of the seven patterns do not inject an error at all** — `slow_tool` sleeps 30 s and then succeeds, `stderr_warning_storm` emits 200 warnings and then succeeds — so a third of the injected matrix tests latency and log-noise tolerance rather than recovery, and results are reported by class and never pooled (Supplementary Table S6)."
- The rescored number replaces the pooled one: 12.5% true-error recovery (95% CI [0.03, 0.32]) against a pooled 41.7%.
- The figure caption tells the reader which columns are which, so the panel cannot be misread: "Columns 8–11 ... inject no error and are latency- and log-noise-tolerance checks rather than recovery tests (Supplementary Table S6); the columns are alphabetical, so the twelfth and rightmost, `wrong_format_output@lofreq`, is a true-error pattern and a green cell there is a genuine recovery."
- Table S6 also documents why the true-error denominator is 8 and not 10: `silent_truncation` and `wrong_format_output` both act on the VCF and `bwa` emits none, so those two combinations are inexpressible — "**by design and not by post-hoc exclusion** ... it was not stated in the first version of this manuscript."

Not addressed by new data. The expanded suite (T3.5) — malformed FASTQ, missing input, permission denied, disk-full, incompatible tool version, stale intermediates, biologically implausible output — was **not run** as a designed matrix. See §5.8.

R3.8 — Statistical honesty at n = 3; replace confidence adjectives with interval estimates

The point. 3/3 supports roughly a 29–100% interval. Either add replicates or soften every claim resting on three trials.

Response. Accepted, and we did both. The result is that the original headline did not survive.

What changed.

- Every proportion carries a Clopper–Pearson exact 95% interval, and the width is stated where a reader might skip it: "3/3 gives [0.29, 1.00], 10/10 gives [0.69, 1.00]."
- Intervals are applied against our own favourable numbers, not only the unfavourable ones: "That saturation should be read against its interval, since 9/9 is [0.66, 1.00], whose lower bound is not separated from several local cells."
- New Results section, *Repeated sampling at the headline condition*: "The n = 3 headline does not survive repeated sampling. Adding 7 seeds to the original 3 at v2 Track A ... nine of 12 executors are 10/10 (exact 95% CI [0.69, 1.00]); `gpt-oss:20b` is 9/10, `qwen3.5:4b`, which scored 3/3 at n = 3, is 7/10, and `nemotron-3-nano:4b` is 1/10 ([0.00, 0.45]). Every model that moved, moved down, which is the expected direction rather than symmetric noise, so we report n = 10 as the headline and treat every n = 3 cell as an interval."
- What n = 10 is and is not: "This is **repeated sampling from a single configuration, not replication**: ten seeds on one machine, one engine build, one quantisation, one plan and one scorer vary the sampler and nothing else."
- The seed extension raised v1g, v1.25 and v1.5 to as many as ten seeds per model (273 new cells, `matrix_jetson_seedext.jsonl`): pooled descriptive totals are 31/126, 63/127 and 109/127. The primary paired model-level analysis gives a mean difference of 0.253 for v1.25 minus v1g (exact sign-flip p = 0.027) and 0.354 for v1.5 minus v1.25 (p = 0.0039). Prose removal remains "not resolved at these sample sizes" (paired sign-test p = 0.63).
- The flagged adjective is gone; Limitations now states that the one contrast still unresolved is prose removal, and that the v1g/v1.25 contrast and the one-to-ten-command-lines step are resolved at ten seeds.

R3.9 — Seed semantics differ between API and local

Folded into C4. For local models, the seed selects a trajectory at fixed decoding parameters (Supplement, *Inference settings*). For the Anthropic API runs, the same section states that "the harness constructs no seed argument at all" and that the recorded integer "is a bare run identifier and is reported as such".

R3.10 — Heatmap cells show n = 2, 3, 6 while the text says three runs per condition

The point. Either the counts or the text is wrong. State explicitly whether failed generations were dropped or reruns pooled, and on what pre-specified rule.

Response. Accepted; correct catch. Nothing was dropped for producing a bad answer, and the varying n has three separate causes, all now stated.

What changed.

- The exclusion rule, stated once in Methods and applied to both arms: "One exclusion rule governs both arms: **cells whose generation ended in truncation, refusal or provider error are scored 0 and retained, and no cell was dropped for producing a bad answer.**"
- The n = 6 and n = 108 cells are explained by pooling, not by exclusion: "the nine cells collapsing to seven plan conditions because Track B occupies three of them", and Table 3's caption — "Track B pools three (plan file, track) cells, hence n = 108."

- The design multiplies out explicitly: "12 local executors × 9 (plan file, track) cells × 3 seeds = 324 generations, and 3 frontier executors × 9 × 3 = 81".
- A pitfall in the released data is documented so re-analysts meet it knowingly: "**Every Track-B run is a no-plan run regardless of which plan file was nominally passed on the command line.**" Grouping runs by plan file without honouring this yields the convincing but false conclusion that the most detailed plan performs worst; `revision/scripts/gradient.py` implements the correct mapping and asserts it.
- Generation outcomes for every block are Table S10, and the row-count denominators are reconciled by `find` over the released tree in Table S2.
- The thirteenth row in Figure 2 is captioned as out-of-sample and excluded from the totals: "the twelve-model gradient totals given in the text exclude `qwen3.8:27b`, which was run separately and is shown here for comparison."
- Where Table 4's Jetson values differ in the third decimal from `SYNTAX_DENSITY.md`, the reason is given in the caption: this manuscript's retention rule scores the one truncated v1g generation 0 and keeps it; that file drops it.

R3.11 — Cross-platform: the same weights should produce equivalent output

The point. Outputs differ across machines running the same model, which the manuscript did not explain.

Response. The premise is incomplete and we answer it with the build facts rather than with compliance; see §3.2 (premise) and §3.1. We do not use the answer to keep a claim we cannot support.

What changed.

- Supplement, *Platforms, engines, and reproducibility metadata*: "Part of the answer to why identical weights do not give identical outputs across them is that they are not the same build of the same weights: the CUDA platforms ran GGUF `q4_K_M` weights under ollama, the Apple-silicon machines ran MLX 4-bit builds." Table S7 gives engine and quantisation per platform.
- Engine configuration that materially changes behaviour is reported: flash attention disabled after 22 illegal-memory-access faults in six hours across 5 of 12 models; `f16` KV cache; one model loaded at a time; 400 earlier runs under a `q8_0` cache discarded rather than pooled.
- The claim is withdrawn rather than defended: "accuracy was not measured comparably across platforms at these sample sizes, so no claim of platform-independent accuracy is made".

Not addressed by new data. The standardised fixed-configuration cross-hardware benchmark (T3.7) was **not run**. See §5.9.

R3.12 — Table 6: the reported median for the 2×A5000 system lies outside its reported IQR

The point. A summary statistic in the table is impossible. Either a transcription error or a bug in the summary script.

Response. Correct, and worse than reported. It was a hand-entry failure in a table whose own caption claimed hand-entry had been eliminated.

What changed. The table (now Table S13) is "**fully generated** by `revision/scripts/table6.py --markdown`", and the generator "asserts min ≤ Q1 ≤ median ≤ Q3 ≤ max before printing". The caption states the full extent of the original defect: "the generator emitted no min–max column, so the 'full range' was hand-entered, four of its five rows were wrong in the direction that understates the maximum — the M4 Pro row by a factor of eight — and the RTX 5080 row printed that platform's IQR in the full-range column while declaring the IQR unreportable. At n = 6 and n = 3 a quartile estimate is weak and should be read as such, but withholding it while reporting a hand-typed range was the worse choice."

A related archival pitfall found during the recomputation is documented.

> "an archived error-matrix log for the M4 Pro has a reduced schema in which the wall-clock field means something else and reads ~2 s; the correct file for that platform is the `_r2` variant."

R3.13 — Honest cost accounting; replace "free" with "no marginal API fee" and build a real cost table with break-even

The point. "Free" is not free. Price the hardware, depreciation, power, and staff time against per-run API cost, and give a break-even run count.

Response. Accepted in full. This was the most useful comment in the review, because doing it properly defeats the paper's original economic motivation, and we now say that in the abstract, the results, the discussion and the conclusion.

What changed.

- Economic claims now use "no marginal API fee" rather than treating local execution as costless; remaining uses of "free" in quotations or model descriptions should not be read as a total-cost claim.
- The full model is Table S9 with measured inputs marked: "Two of the cost model's eight inputs are measured: frontier-API cost of $0.0662 per run over 81 API cells, and Jetson **wall-clock** time of 86.7 s per run ... The other six are assumptions exposed as parameters in `revision/scripts/cost_model.py`."
- Break-even and annualised crossover: "Against a three-year cost of ownership of $5,045, cost recovery arrives at 76,424 runs, which is 15.3 years at 5,000 runs per year and therefore not a break-even under a three-year horizon. The like-for-like **annualised** comparison gives a fixed local cost of $1,682 per year, equalled by API spend at **25,475 runs per year**".
- The asymmetry R3 implied is corrected in the reviewer's favour: "Charging labour to the local arm alone is asymmetric, since the API route also needs setup and maintenance; adding an API-side labour term moves the annualised crossover to 17,900 runs/year at 4 h/year and 10,325 at 8 h/year, and at 16 h/year the local box is cheaper at any run rate."
- The headline conclusion is stated against our own interest: "**The economic case for a local executor is made by agentic workloads, not by the single-pass task this paper originally rested on**", and the section is titled *Cost: a local executor does not pay for itself on a task this small.*
- Supervision labour is excluded and the exclusion is flagged: "Supervision labour is not priced at all and benchmark 2 shows it is not zero, so **the cost model is the zero-touch case, which benchmark 2 did not achieve.**"
- The Discussion's remaining argument for local execution is no longer cost: "At the task sizes measured here the reasons to run an executor locally are data governance and freedom from per-call metering, not cost."

R3.14 — Figure numbering is broken; audit every cross-reference

The point. Figures are labelled and cited inconsistently, and the on-disk filenames imply a third ordering.

Response. Accepted. Every cross-reference was audited, and because Genome Research places Methods after Results, we embedded an explicit manifest rather than leaving a typesetter to infer the mapping from filenames that no longer match.

What changed. A build note in Methods carries the manifest: Figure 1 → `rev_fig0_design.png`; Figure 2 → `rev_fig1_gradient_thinkoff.png`; Figure 3 → `rev_fig4_n3_vs_n10.png`; Figure 4 → `ms_fig3_qwen3p6_27b_error.png`; Figure 5 → `galaxy_demo_counts.png`; Figures S1–S4 similarly. The note states why it exists: "the archived filenames no longer match the in-text numbering and a typesetting pass must not infer one from the other."

R3.15 — De-promotionalize

The point. "The winner", "protagonist", "all you need is a plan", "pick the cheapest box" and similar do not belong in a paper.

Response. Accepted. All are removed, and one framing was removed for being underpowered rather than for tone.

What changed.

- No model is named a winner. Where a per-model result is favourable it is reported with its interval and its caveat: nine of twelve are 10/10, one is 9/10, one is 7/10 and one is 1/10.
- "All you need is a plan" is contradicted by the paper's own scope statement (C2) and by the transcription result.
- "Pick the cheapest box" is contradicted by the cost section (R3.13).
- The generational framing was removed for lack of power, not for tone: "The out-of-sample pair `qwen3.8:27b` against `qwen3.6:27b` is underpowered rather than null — 8 of 27 matched cells discordant, splitting 6 to 2 in favour of the *older* model ($p = 0.29$) — so the generational framing has been removed from the abstract and the conclusion."
- The claim about executor class is stated in a form that names its own confounds: "Executor class — 4-bit open-weight models of at most 35 B against a commercial API — therefore determined how much specification an executor needed, with quantisation and vendor confounded with weight availability."

R3.16 — Reproducibility metadata: model IDs and digests, quantization format, engine versions, tool versions, full command lines

The point. Model family names are not enough.

Response. Accepted. The table is script-generated so that it cannot drift, and generating it exposed a parser fault we report rather than silently fix.

What changed.

- Table S8 is "Collected by `revision/scripts/collect_repro_metadata.py` into `revision/repro_metadata.json` and rendered with its `--markdown` flag, so the table is regenerated rather than transcribed", recording per model the exact tag, blob digest, architecture, parameter count, quantisation format, context length, declared capabilities and default decoding parameters.
- Environment: ollama 0.32.5; JetPack 5.1.2, CUDA 11.4, compute capability 8.7; bwa 0.7.18, samtools 1.21, bcftools 1.21, htslib/tabix 1.21, lofreq 2.1.5.
- Coverage was wrong and is fixed: "**This table contains all fourteen local models the study uses.** The first version of it contained twelve and omitted `gemma4:12b` … and `qwen3.8:27b`, which is the benchmark-2 dense arm".
- Why the omission mattered is stated: without that row, "omitting that row left the sampling configuration of the case-study executor unknown to a reader."
- The parser fault is disclosed: "a multimodal tag carries a `Projector` block whose own `architecture` and `parameters` keys were being filed as the model's, so `qwen3.8:27b` first collected as `clip` at 460.73 M. The fault is fixed in `collect_repro_metadata.py`, it changed none of the twelve original rows, and it is recorded here because a script-generated table is only as trustworthy as its parser."
- A residual provenance gap is stated as a reproducibility failure of the artifacts: "Each cell's `meta.json` records the plan file path and plan name it was rendered from, but **not** a git revision for that file … Adding a plan blob SHA to `meta.json` would close this and was not done."
- The token-budget choice, previously unjustified, is now justified and probed: 16,384 tokens is uniform across local models and between one eighth and one sixty-fourth of declared native context, with a sensitivity ablation reported in Results.

4.5 Additional R2 detail items carried in the source markup

R2's review contained finer-grained items beyond R2.1–R2.11. We tracked them as R2.12–R2.22 in the source comments so each has a locatable home. In brief:

- R2.12: the study-design figure and the governance boundary drawn on it (Figure 1).
- R2.13: the cell ledger and how the design multiplies out (Methods, *Study design*; Tables S1–S2).
- R2.14: the post-hoc authorship of v2 (Introduction; Methods, benchmark 2).
- R2.15: external comparators and their protocols (Introduction; Supplement).
- R2.16 and R2.20: repeated sampling and the generational contrast (Results, *Repeated sampling*).
- R2.17: the cost model's measured versus assumed inputs (Methods; Results, *Cost*).
- R2.18: the fabrication study's design and its excluded launches (Methods; Results, *Fabrication*).
- R2.19 and R2.22: wall-time dispersion by platform (Table S13).
- R2.21: disclosure of LLM assistance in harness construction, analysis scripting and manuscript drafting (Acknowledgements), and the reasoning-arm budget accounting (Results, *Enabling reasoning*).

R2.3b is the residency half of R2.3, dispositioned in §3.3.

4.6 Tier-3 experiments that were run

T3.3 (R3.5) — Exploratory vs confirmatory separation

Partly done, and the shortfall is reported rather than glossed. The existing gradient is declared exploratory in the Introduction. A frozen protocol was then written for the held-out workflow — the verbatim executor-selection rule quoted under R3.5 — and it was violated in two ways we state in Methods: the dense arm actually run was `qwen3.8:27b` rather than the model the rule returned, "so **that arm is exploratory rather than confirmatory.**", and "Plan sufficiency was never confirmatory on either benchmark, because the Galaxy plan was corrected repeatedly while benchmark 2 ran." The confirmatory claim R3 offered to rescue is therefore not rescued, and we do not claim it.

T3.4 (R3.3, R3.2, C2) — Second biological benchmark

Run, without perturbations. The second workflow class is per-sample RNA-seq quantification of *C. auris* through the 30-step IWC `rnaseq-pe` workflow on usegalaxy.org (Results, *Case study*; Figure 5; Table S14). Both executors submitted a parameter-correct invocation — 2 of 2 inputs and 8 of 8 parameters — and the workflow reproduced the published quantification at 5,594 genes and 93.1–93.6% assigned of uniquely mapped fragments.

R3 suggested combining T3.3 and T3.4 in one workflow, which we did. R3 listed four data-perturbation options: varied depth and allele frequency, a reference-genome swap, a tool-version change, an external API call. The four were **not** run. Limitations states the consequence.

> "No *data*-perturbation series was run — no variation in depth or allele frequency, no swap of the reference sequence itself, no tool-version change — which is what would let the accuracy metrics separate a good caller from a bad one rather than a wired pipeline from an unwired one; the perturbed-*plan* pass moved a path, not the biology."

The three caveats that must travel with the case-study result are in the main text.

> "the plan supplied the invocation body literally, a human pre-built two of the seven plan steps' inputs after a transport fault, and neither executor completed the final reporting step."

T3.6 (R3.8) — More replicates

Run at the headline condition, then at the discriminating conditions. n was raised from 3 to 10 at plan v2, Track A, reasoning off, by adding seven seeds and pooling (Figure 3, Table S11). Three of twelve models moved and all three moved down. The remaining discriminating cells — v1g, v1.25 and v1.5 — were then raised to as many as ten seeds per model after the second review cycle (273 new cells, `matrix_jetson_seedext.jsonl`, Supplementary Table S16). Pooled descriptive totals are 31/126, 63/127 and 109/127. In the primary analysis treating models as paired units, the mean differences are 0.253 for v1.25 minus v1g (exact sign-flip $p = 0.027$) and 0.354 for v1.5 minus v1.25 ($p = 0.0039$). The prose-removal contrast (v2 against v1.5) was not extended and remains unresolved.

---

5. Experiments that were not run, and three that were subsequently completed

We list these together so that no reader has to reconstruct what is missing from prose. Three entries from the previous version of this list have since been run. They are the seed extension at the discriminating conditions (§4.6, T3.6), one variant of the perturbed-plan control (§5.3 below), and the bounded repair arm (§5.1 below). The remainder were not run, and no result in the manuscript depends on any of them. They are tabulated in main-text Table 8, captioned "Experiments that would each resolve a claim this paper states conditionally. None was run".

5.1 T3.1 — Bounded iterative repair arm (C1, R2.10, R3.1)

Run, after the second review cycle, at the perturbed condition. The previous version of this letter declined this arm for its harness cost. It has now been run where it matters most: against the perturbed plan, on which every transcriber fails. The design: if the script exits nonzero, the model sees its own script, the exit code and the last 40 lines of the execution log, and may submit a fix. At most three attempts are allowed. The retry signal is the exit code only; the score is computed afterward and never shown. Thirteen models × 3 seeds (`revision/logs/matrix_jetson_repair.jsonl`; per-attempt scripts under `revision/runs_repair/`).

The result has three tiers. The two binders were perfect on attempt 1, unchanged. `qwen3.6:27b` and `qwen3.8:27b` were perfect on attempt 2 in all three seeds; `laguna-xs-2.1` in two of three. The remaining eight models recovered 0 of 24 seeds in three attempts. Repair rescued 8 of 33 copier seeds. Class success with repair is 14/39, against 6/39 one-shot and 34/36 unperturbed (Results, *Three attempts separate transcribers that repair from transcribers that do not*; main-text Table 7; Supplementary Table S18).

What the arm concedes: the single-pass constraint accounts for part of the transcription finding. For three models, one look at the error replaced the current path the plan lacked. `qwen3.6:27b`'s second attempt tests for the old path, falls back to the path stated in the prompt, and proceeds. What it does not concede: that feedback fixes the class. Eight models failed all three attempts, with the error naming the missing file and the correct path in their prompt. They were not idle. They changed flags, loops and index names while keeping the dead path. Two guessed `data/ref/rCRS.fa` on attempt 3 — the correct filename from the prompt, in the wrong directory. Still not run: repair at the lean plans v1 and v1.25, and any frontier arm. The arm ran at three seeds, under one perturbation type, with a retry only on a nonzero exit.

5.2 T3.2 / R3.6 — Human-expert and second-frontier planner arms

Not run. The third arm R3 asked for, a plan assembled from official tool documentation, *is* answered: v1g is a `lofreq` block extracted mechanically from the Galaxy IUC registry at commit `39e7456`, and it behaves as its content predicts rather than as its authorship would. The other two arms — an expert-written human plan and a second frontier model's plan, held at matched detail — were not run. Why: matching detail level across authors is the hard part, and v1g demonstrates the difficulty concretely, since its syntactic density is not matched to v1.25's despite both adding one invocation. Running the arms without solving the matching problem would produce a comparison that confounds author with density, which is the confound the arms exist to remove. What it would take: two additional plans, roughly 40 further cells each at three seeds, plus a defensible density-matching procedure agreed before the plans are written.

5.3 R2.1 / R3.preamble — The v2-shifted anti-copy control

Run, in one variant, after the second review cycle. The previous version of this letter called this the single most valuable missing experiment in the paper; it has now been run in its moved-reference form: plan v2 verbatim, the reference at `data/ref/GRCh38_chrM/rCRS.fa` instead of the plan's literal `data/ref/chrM.fa`, the real path stated in the prompt's DATASET section, the sandbox validated with a hand-written copier (fails) and binder (succeeds) before any model ran. Thirteen local models × 3 seeds: **6/39 perfect against a 34/36 control, and every model is 3/3 or 0/3.** Eleven of thirteen models copied the stale path verbatim and died at the first step; `gemma4:31b` and `gpt-oss:20b` rebound every path and stayed perfect. The result is the manuscript's central finding (Results, *A one-path perturbation separates transcription from binding*; Table 6; Supplementary Table S17). Still not run: the other perturbation variants — renamed samples, an extra sample, permuted step order, a wrong output path — and any frontier arm, for which no API spend was authorised; these are the first row of main-text Table 8.

5.4 R2.6 / C4 — The pinned `top_p` / `top_k` re-run

Not run. Why: the confound was found late in the revision, and a partial re-run would have produced a manuscript in which some cells were decoded under a pinned decoder and others were not, which is worse than a uniform disclosure. What it would take: 156 cells — the headline cell at 12 models × 10 seeds and the discriminating cell at 12 models × 3 seeds — roughly 4 GPU-hours against the 75.27 h already spent, "and would put the pinned numbers in the headline with the current run as a sensitivity check." Consequence, stated in the manuscript: every cross-model ordering here is provisional. The within-model gradient is unaffected.

5.5 R3.2 — The deterministic templater baseline

Not run. Why: on the inputs this study used, the outcome is not in doubt — a 20-line templater will produce a correct pipeline every time — so the comparison is only informative on *perturbed* inputs. The perturbed-plan pass (§5.3) now supplies the perturbed side of that comparison without the templater itself: eleven of thirteen local models behaved exactly as a fixed template would — reproduced the stale path and failed — and two did what no template can, rebinding the paths. The manuscript states this in the Discussion: "For most of this model class, then, the objection stands: the model adds nothing a template does not. For two models it demonstrably adds the one thing that matters." The templater itself was still never scored beside the models; what it would take, from Table 8: "no GPU", plus the perturbed input set.

5.6 R3.3 / C2 — A plan gradient inside benchmark 2

Not run. Benchmark 2 exercises one plan, corrected repeatedly while it ran. It therefore illustrates plan sufficiency and does not test it, which is stated in the Results and again in the Discussion: "a two-point plan gradient in the loop would make it test plan sufficiency instead of illustrating it, and was not run." Why: the Galaxy runs are multi-hour, involve a shared production server with real queue times, and required repeated human intervention; a gradient would multiply that cost by the number of conditions with no replicates available at any of them. What it would take: at minimum two plan conditions at the leanest and most detailed ends, with several launches each, on a server whose queue behaviour is outside our control.

5.7 T3.8 / R2.9 / C5 — Biomni head-to-head

Not run, and we argue it should not be the bar for this paper. Biomni is an open-ended tool-using research agent evaluated across a wide biomedical task set including literature retrieval and experimental design; this paper measures how much specification a fixed-hardware executor needs to bind and run a pre-specified pipeline. Those are different problem framings with different independent variables — Biomni's evaluation varies the task, this one varies the plan — and a head-to-head would require choosing whose task set to adopt, which decides the answer before the comparison begins. R2's own comment concedes that the design concerns "may not need to be in the scope of the article".

We do not offer this as a reason to have no external anchor. We do not pretend the positioning discussion substitutes for one. The manuscript states the gap in its sharpest form and names the cheaper experiment that would close it.

> BixBench "ships 53 public Dockerised capsules precisely so that new executors can be placed on a common scale, and running this study's executors on a subset of them is a bounded experiment on public artifacts that would convert this declination into an anchor. It was not run, and this is the sharpest reproducibility gap in the paper's positioning."

5.8 T3.5 / R3.7 — Expanded error-injection suite

Not run as a designed matrix. The relabelling R3 asked for was done and changed the headline number (12.5%, not 41.7%). The new patterns — malformed FASTQ, missing input, permission denied, disk-full, incompatible tool version, stale intermediates, and syntactically valid but biologically implausible output — were not added. Why: the last of these is the scientifically interesting one and it is also the one that needs a truth-set design of its own, since "biologically implausible" has to be operationalised against something; the others are cheap but would extend a matrix whose central finding is already that the outcome is pattern-determined rather than model-determined, and whose per-cell artifacts were not archived. What it would take: seven new PATH shims, re-archiving of per-cell artifacts, and roughly the same cell count as the existing matrix per executor–plan pair.

5.9 T3.7 / R3.11 — Standardised cross-hardware benchmark

Not run. Fixed prompt, fixed output length, fixed repetitions, matched model configuration on each of the five machines, reported as tokens/s with dispersion. Why: three of the five platforms are not co-located and two run a different inference engine entirely, so "matched configuration" requires deciding what counts as matched across GGUF `q4_K_M` and MLX 4-bit, which is the same problem the reviewer's premise raises. What we did instead: reported the mixed-workload timing with dispersion from a generated table (Table S13), and withdrew the accuracy claim the comparison was being used to support.

---

6. Summary of what the revision costs the paper

We think it is worth stating plainly which of our own claims did not survive this revision, because several did not.

1. **The economic argument fails at this task size.** Recovery arrives at 76,424 runs against a three-year cost of ownership of $5,045; the annualised crossover is 25,475 runs/year before any API-side labour term. Cost is no longer offered as a reason to run locally.
2. **The n = 3 headline did not survive n = 10.** Three of twelve models moved, and all three moved down.
3. **The single-pass recovery rate is 12.5%, not 41.7%.** Two of seven injected patterns inject no error.
4. **The run-level metric is degenerate on the local arm.** M3 takes only 0.0 and 1.0 across all 324 local reasoning-off cells. Precision is 1.000 by construction.
5. **No Galaxy executor completed the reporting step.** Neither arm reported a single per-sample number in the two runs of step 7; a human read the six values out of the server. "Both executors submitted a correct invocation" is true; "both executors succeeded" is not.
6. **The headline finding is close to a restatement of the task-selection criterion.** Both tasks were chosen for having no data-dependent parameter choices. The finding is that a plan works when it states the parameters.
7. **What was measured at the detailed plan is, for most models, copying. The experiment that separates copying from binding was run and says so.** One moved path, stated in the prompt, sent eleven of thirteen local models from a 34/36 condition to 0. The original headline model is

among the eleven. Two models bind. With the error fed back, three of the eleven copiers repair within three attempts and eight do not (§5.1). The title no longer claims binding for the class.

8. **Cross-model orderings are provisional**, because `top_k` and `top_p` sat at differing per-model defaults.
9. **Four artifact classes are not released**: the Galaxy plan files, the fabrication study's code and per-cell outcomes, the defect ledger with its diffs, and the `loom` session logs. Several benchmark-2 counts therefore rest on a hand-typed contemporaneous record. They are marked as such at every point of use.

What survives is narrower than the first submission claimed and, we think, more useful. A small local 4-bit model reliably produces a working pipeline when the plan hands it invocations it can copy verbatim. For eleven of the thirteen models tested, "copy" is literal: the plan works only while its paths match the data, because the model transcribes rather than binds. Two models bind. Three more repair when shown the error, and eight do not. The explanation around the invocations adds nothing detectable at these sample sizes. Frontier executors need none of it. And the reason to run an executor locally is data governance rather than cost.